

KitchenCo

Backend Architecture

Target architecture for the MVP backend — Supabase (Postgres + Auth + Storage + Edge Functions), replacing the current mock, in-memory Expo frontend.

Product	Catering / meal-ordering platform — individual & corporate accounts
Scale	MVP launch — hundreds of users, one kitchen, tens-hundreds of orders/day
Stack	Supabase (Postgres, Auth, Storage, Edge Functions, Realtime)
Integrations	PayFast, transactional email, SMS/WhatsApp, push notifications
Prepared	September 2026

Status: design doc — no backend exists yet. The current Expo/React Native frontend makes zero network calls; all state is in-memory React state plus a handful of AsyncStorage keys. This document is the target architecture to replace that with a real backend.

Why Supabase over Firebase

The domain is inherently relational, not document-shaped:

- Corporate **batch status** (one status per company + delivery-day, propagated to every order in that batch) needs an atomic multi-row transaction — trivial in Postgres, awkward in Firestore.
- Reporting (revenue by category, top sellers, trend buckets) is `GROUP BY / SUM` — native SQL, would mean hand-rolled aggregation or Cloud Functions in Firestore.
- Invoice numbers need a real atomic sequence — Postgres `SEQUENCE`, not a Firestore counter-with-race-condition workaround.
- Row Level Security maps cleanly onto "own orders / own company / admin sees all" without writing security rules in a separate DSL.
- Same TypeScript/JS stack end-to-end (Edge Functions are Deno/TS), and Postgres is a portable escape hatch if you ever outgrow the BaaS.

1. System Architecture

Expo RN app (client)

- supabase-js SDK → Supabase Postgres (PostgREST auto-API, RLS-gated)
 - direct CRUD: menu browsing, addresses, order history, company signup lookup, cart is client-local until checkout
 - RPCs (Postgres functions) for anything transactional/
business-rule-bound: `place_order`, `update_order_status`, `submit_order_rating`, `report_order_dispute`
- Supabase Auth → email+password, real hashing, role/account_type read from a 'profiles' table (never from client-editable JWT claims)
- Supabase Realtime → subscribes to 'orders' (own rows / own company) for live status updates, replacing manual refresh
- WebView (PayFast) → PayFast hosted payment page (redirect only, no card data touches our stack)

Supabase Postgres

- triggers/functions → batch status propagation, invoice number minting, cutoff/lead-time + delivery-fee validation (server-side, never trust client-computed totals)
- DB Webhooks → fire on order insert / payment_status change

Supabase Edge Functions (Deno/TS) - anything needing secrets or trusted server execution

- `create-payfast-payment` generates the signed PayFast payload server-side (merchant key + passphrase NEVER ship to the client)
- `payfast-itn` PayFast's server-to-server webhook; validates signature, flips `orders.payment_status` - this, not the client redirect, is the authoritative "payment succeeded" signal
- `notify-order-event` triggered by DB webhook; fans out email (Resend) + push (Expo Push API) + SMS/WhatsApp (Twilio) on status/payment changes
- `send-production-sheet` admin-triggered; emails the chef PDF via Resend (replaces device mail-composer dependency)
- `pg_cron jobs` discount expiry sweep, materialized view refresh

Sync flows: menu browsing, address CRUD, order history reads — direct PostgREST calls under RLS.

Transactional flow (place_order RPC): client sends cart + delivery address id + payment reference. Server re-validates the 9AM cutoff / 2-3 business-day lead time, re-resolves delivery fee from the stored distance_km, re-applies discount/subsidy, mints an invoice number, inserts orders + order_items in one transaction, returns the created order. Client never gets to dictate price.

Async flow (payment): client → PayFast hosted page → PayFast → `payfast-itn` webhook (server-to-server, out of band) → `orders.payment_status` flips → DB webhook fires `notify-order-event` → email/SMS/push sent → client sees the update via Realtime subscription, not by trusting its own WebView redirect.

Batch status propagation: `update_order_status(order_id, new_status)` is a **SECURITY DEFINER** Postgres function, admin/chef-role only, that resolves `batch_key = company_id || delivery_date` and updates every non-

cancelled sibling order in one statement — this must live in the DB layer, not app code, so it can't be bypassed by a direct table write.

Caching: menu/cycle-menu data changes rarely — short client-side TTL cache, invalidated on admin edit.

Dashboard/analytics aggregates come from materialized views refreshed on a schedule (pg_cron), not live joins, so the admin dashboard doesn't do heavy aggregation on every load as order volume grows.

2. Project / Folder Structure

Supabase projects aren't a conventional Node server tree — the "backend" is mostly declarative (SQL migrations + RLS policies) plus a small set of Edge Functions. Proposed layout, sitting alongside the existing `Kitchen_APP/` Expo project:

```
supabase/
├── migrations/          # timestamped SQL migrations - schema is version-controlled here
│   ├── 0001_init_schema.sql
│   ├── 0002_rls_policies.sql
│   └── ...
├── functions/
│   ├── create-payfast-payment/  # signs the PayFast payload server-side
│   │   └── index.ts
│   ├── payfast-itn/            # PayFast webhook receiver, signature-verified
│   │   └── index.ts
│   ├── notify-order-event/      # email + push + SMS fan-out on order events
│   │   └── index.ts
│   ├── send-production-sheet/   # chef PDF email send
│   │   └── index.ts
│   └── _shared/                 # shared Deno helpers (payfast signing, resend client, cors)
│       ├── payfast.ts
│       ├── email.ts
│       └── push.ts
├── seed.sql                # demo data mirroring current mock (companies, menu, admin user)
└── config.toml              # local dev config (Supabase CLI)

Kitchen_APP/
├── src/
│   └── lib/
│       └── supabase/
│           ├── client.ts        # supabase-js client init
│           ├── orders.ts        # typed wrappers around place_order/update_order_status RPCs
│           ├── menu.ts          # typed menu/cycle-menu queries
│           └── types.ts          # generated DB types (supabase gen types typescript)
```

One-line purpose per piece

- `migrations/` — the actual schema, reviewed and applied like code, not clicked together in a dashboard.
- `functions/*` — the only place secrets (PayFast passphrase, Resend/Twilio API keys) exist.
- `functions/_shared/` — avoids duplicating PayFast signing/email logic across functions.
- `seed.sql` — lets a fresh dev/staging DB come up looking like the current demo data.
- `src/lib/supabase/` — the frontend's only door into the backend; screens never call supabase-js raw so RPC contracts stay in one place.

3. Database Schema (Postgres)

All money stored as integer cents (`_cents` suffix) to avoid float rounding on currency. All tables have `created_at timestampz default now()` ; omitted below for brevity.

Table	Key columns	Notes
profiles	<code>id</code> uuid PK → <code>auth.users</code> , <code>email</code> , <code>name</code> , <code>role</code> enum('customer','admin') , <code>account_type</code> enum('individual','company') , <code>company_id</code> → <code>companies</code> , <code>company_address_id</code> → <code>company_addresses</code>	1:1 with <code>auth.users</code> ; role is never in the JWT, always read server-side
companies	<code>id</code> uuid PK , <code>name</code> , <code>meal_subsidy_cents</code> int null	
company_domains	<code>id</code> , <code>company_id</code> → <code>companies</code> , <code>domain</code> unique	separate table (not an array) so signup/login can do an indexed lookup, re-checked on every login per current business rule
company_addresses	<code>id</code> , <code>company_id</code> → <code>companies</code> , <code>label</code> , <code>street</code> , <code>unit</code> , <code>suburb</code> , <code>city</code> , <code>code</code> , <code>instructions</code> , <code>distance_km</code> numeric , <code>is_primary</code> bool , <code>sort_order</code> int	
addresses	<code>id</code> , <code>user_id</code> → <code>profiles</code> , <code>label</code> , <code>street</code> , <code>suburb</code> , <code>city</code> , <code>code</code> , <code>distance_km</code> numeric , <code>is_default</code> bool	individual users only
menu_categories	<code>id</code> , <code>name</code> , <code>sort_order</code> , <code>icon</code>	
menu_items	<code>id</code> , <code>category_id</code> → <code>menu_categories</code> null , <code>name</code> , <code>description</code> , <code>image_url</code> , <code>tags</code> text[] , <code>active</code> bool , <code>source</code> enum('static','cycle')	
menu_item_sizes	<code>id</code> , <code>menu_item_id</code> → <code>menu_items</code> , <code>label</code> , <code>price_cents</code>	
menu_item_addons	<code>id</code> , <code>category_id</code> → <code>menu_categories</code> , <code>name</code> , <code>price_cents</code>	
cycle_menu_slots	<code>id</code> , <code>week_number</code> int check 1-8 , <code>day_of_week</code> enum(Mon..Fri) , <code>slot</code> enum(main,vegetarian,healthy,curry,gourmet_sandwich) , <code>item_name</code> , <code>description</code>	flat-priced via <code>pricing_config</code> , not per-row
cycle_rotation_config	singleton row: <code>anchor_monday</code> date , <code>week_offset</code> int	mirrors existing <code>getCycleWeekForDate</code> logic exactly
pricing_config	<code>key</code> text PK , <code>value_cents</code> int	e.g. <code>cycle_item_price</code> , editable without a redeploy
discounts	<code>id</code> , <code>code</code> unique , <code>percentage</code> , <code>active</code> , <code>expires_at</code> , <code>company_id</code> null , <code>category_id</code> null , <code>item_id</code> null	nullable targeting fields = global scope, matches current mutually-exclusive-ish targeting

Table	Key columns	Notes
orders	id, invoice_number unique, user_id → profiles, company_id null, delivery_date date, delivery_address_snapshot jsonb, delivery_fee_cents, subtotal_cents, discount_id null, discount_amount_cents, subsidy_amount_cents, total_cents, status enum(pending, preparing, on_the_way, delivered, cancelled), payment_status enum(pending, paid, failed, refunded), payment_reference, notes, batch_key (generated column)	batch_key = company_id + delivery_date, is what update_order_status groups on
order_items	id, order_id → orders, menu_item_id null, name (snapshot), price_cents, quantity, size_label, notes, delivery_date date	notes = allergies/special requests — chef sheet needs this verbatim
order_ratings	order_id PK → orders, rating smallint check 1-5, feedback text, submitted_at	only insertable when orders.status = 'delivered'
order_disputes	order_id PK → orders, reported_at, reason, ticket_ref unique, status enum(investigating, refunded, resolved)	ticket_ref generated server-side (sequence-backed), not Math.random()
saved_cards	id, user_id → profiles, cardholder_name, last4, card_type, expiry_date	last-4 only, ever — real PAN never touches this DB
announcements	id, title, body, company_id null (null = broadcast), sent_at, created_by	
push_tokens	user_id → profiles, expo_push_token, platform, updated_at	needed for real push (currently local-only notifications)

Indexes: orders(batch_key), orders(user_id, created_at desc), orders(delivery_date, status) (chef production-sheet query), order_items(order_id), company_domains(domain) unique, discounts(code) unique, menu_items(category_id, active).

Why SQL, concretely tied to this app: atomic batch status updates across N rows, real GROUP BY reporting (revenue by category/top sellers/trend), FK integrity between orders/companies/addresses, and a real sequence for invoice numbers (current client code openly documents its invoice number as "not SARS-registered" — this must be fixed before going live if VAT invoices are a legal requirement in your jurisdiction).

4. API Design

Supabase auto-generates a REST (PostgREST) and GraphQL API from the schema, gated by RLS — so most of "API design" here is which tables are directly queryable vs. which operations must go through an RPC.

Direct table access (PostgREST, RLS-gated)

Table	Read	Write
menu_categories, menu_items, menu_item_sizes, menu_item_addons, cycle_menu_slots, pricing_config	public (incl. anon, for browsing before signup)	admin only
company_domains (name/id only)	public (signup domain check)	admin only
addresses	own rows only	own rows only
company_addresses	rows for auth.company_id only	admin only
profiles	own row; admin: all	own row (limited fields); admin: all
orders, order_items	own rows only (user_id = auth.uid())	no direct INSERT/UPDATE — via RPC only
order_ratings, order_disputes	own order only	via RPC only
saved_cards	own rows only	own rows only
announcements	company_id = auth.company_id() OR company_id IS NULL	admin only
discounts	public (active ones, for auto-apply)	admin only

RPCs (supabase.rpc(...), business logic that can't be plain CRUD)

Function	Auth	Purpose
<code>place_order(cart, address_id, payment_reference)</code>	authenticated	server-side cutoff/lead-time check, fee/discount/subsidy resolution, invoice minting, atomic insert
<code>update_order_status(order_id, new_status)</code>	admin/chef	propagates to the whole batch atomically
<code>submit_order_rating(order_id, rating, feedback)</code>	own order, status=delivered	
<code>report_order_dispute(order_id, reason)</code>	own order	mints ticket_ref server-side
<code>resolve_company_for_email(email)</code>	public (signup/login)	domain → company lookup, re-run on every login
<code>get_production_sheet(delivery_date)</code>	admin/chef	server-side aggregation mirroring the current client logic (per-client rows + category totals + grand total)
<code>get_dashboard_metrics(period)</code> , <code>get_revenue_trend(...)</code>	admin	backed by materialized views, not live joins

Edge Functions (plain HTTP, outside PostgREST)

Endpoint	Auth	Purpose
POST /functions/v1/create-payfast-payment	authenticated	builds + signs the PayFast form fields server-side; client never sees the merchant passphrase
POST /functions/v1/payfast-itn	PayFast only (signature-validated)	authoritative payment confirmation
POST /functions/v1/send-production-sheet	admin	emails the chef PDF via Resend
(internal) notify-order-event	invoked by DB webhook	email/SMS/push fan-out

5. Auth & Security

- **Auth:** Supabase Auth, email+password with real hashing (current app has *no* password verification at all — this is a straight replacement, not an enhancement). Admin is a `role` column set by an existing admin, never a hardcoded email like the current `admin@gmail.com` check.
- **Authorization:** Row Level Security on every table; role/company checks via `SECURITY DEFINER` helper functions (`is_admin()`, `current_company_id()`) so policies stay short and consistent. Client JWTs carry only `auth.uid()` — role and company membership are always looked up server-side from `profiles`, never trusted from a client-editable claim.
- **Rate limiting:** Supabase Auth has built-in throttling on login/signup. Put Edge Functions (especially `payfast-itn` and `resolve_company_for_email`) behind basic app-level rate limiting (Upstash Redis token bucket, or Cloudflare in front of the project) — light-touch for MVP, tighten if abuse shows up.
- **Input validation:** every RPC re-validates and re-computes server-side (cutoff time, lead-time days, delivery fee from stored `distance_km`, discount eligibility) — the client's numbers are never trusted, only used for optimistic UI. Edge Functions validate the PayFast ITN payload against PayFast's documented signature algorithm before acting on it.
- **Secrets:** PayFast passphrase, Resend/Twilio API keys live only in Supabase Edge Function secrets — never in the Expo bundle (anything shipped in the app bundle is public, including current env-style constants in `payfast.tsx`).

Critical fix vs. current code — PayFast security: the current client hardcodes `MERCHANT_KEY` and builds the payment form directly in the app. In production, the merchant passphrase and signature generation move to `create-payfast-payment` (Edge Function) — the client only ever receives a ready-to-submit, already-signed payload or a redirect URL. Payment success is decided by the `payfast-itn` webhook (signature-verified server-to-server call from PayFast), not by watching a WebView's URL bar.

6. Production Readiness

Error handling: RPCs raise Postgres exceptions with typed error codes (`CUTOFF_PASSED` , `UNDELIVERABLE_DISTANCE` , `INVALID_DISCOUNT` , `DUPLICATE_TICKET`) that the client maps to user-facing copy, instead of generic failures. Edge Functions return structured `{error: {code, message}}` JSON.

Logging/monitoring: Supabase's built-in Postgres + Edge Function logs for infra-level visibility; add Sentry (or similar) in both the Expo app and Edge Functions for exception tracking, with an alert specifically on payfast-itn failures (it's the one path where a silent failure directly costs revenue or leaves an order unpaid-but-placed).

Environment config: separate Supabase projects for dev/staging/prod. PayFast sandbox credentials in dev/staging, the client's real live merchant credentials in prod only (swap-in — no new PayFast signup needed). Secrets via Supabase's per-project Edge Function secrets and Expo's eas.json/app.config env, never committed.

What to build now vs. defer

Now (MVP)	Defer until it's actually needed
Postgres schema + RLS above	Read replicas / sharding — single Postgres instance comfortably handles tens of thousands of orders
place_order / update_order_status / rating / dispute RPCs	Real geocoding/routing for delivery distance — manual distance_km entry matches current behavior and is fine at one-kitchen scale
create-payfast-payment + payfast-itn with real signature verification	Full ticketing-system integration (Zendesk/Freshdesk) for disputes — keep email-based, current mailto becomes a real Resend email
Transactional email (Resend) for order confirmation/status	Power BI / dedicated analytics warehouse — code already has a comment flagging this as a contractual SLA item; defer until reporting need is proven
Expo push notifications for status changes	SMS/WhatsApp — build via Twilio (covers SA numbers + WhatsApp Business API) but gate behind a feature flag until volume justifies per-message cost
Materialized views for dashboard aggregates	Redis/caching layer — defer until menu or report read load actually requires it
Real sequential invoice numbers (Postgres sequence)	Server-rendered PDFs — client expo-print is fine at MVP scale; revisit only if invoices must be tamper-proof or numbering fails a compliance check